

A Fungal Search Algorithm for Map Generation

Elliot McMillan

December 9, 2018

Biologically-inspired algorithms, algorithms that are based on naturally-occurring processes, have been making headway in computer science research. Though many are not as efficient as established man-made algorithms, they are notable for exhibiting new and previously unconsidered ways of solving problems. One such biologically-inspired algorithm is based on the growth patterns of fungi. Fungal search algorithms, or FSAs, have been shown to effectively search mazes, capable of finding a path from beginning to end. This paper proposes a variant of the fungal search algorithm derived from the maze exploration FSA intended to generate mazes with passable routes from beginning to end.

1 Introduction

The fungus kingdom contains some truly unique organisms, notable especially for their growth patterns. Though they were once thought to be plants, fungi have a physiology that is neither plant nor animal. Despite their sedentary appearance, fungi use efficient natural algorithms to expand and locate habitable spaces within their environment, resulting in organisms that are nearly impossible to permanently eradicate.

Fungi are known to have strong foraging capabilities. This capacity translates well into search algorithms. Previous research has explored the fungal search algorithm's ability to search mazes and efficiently navigate from the start of a maze to the end. This paper discusses my subsequent implementation of the algorithm to generate maps of variable size.

This paper is organized as follows: Section 2 describes the natural fungal strategy on which the fungal search algorithm is based. Section 3 describes the general-purpose fungal search algorithm, which Koen and Engelbrecht adapted for maze exploration [2]. Section 4 details my implementation of the algorithm, adapted to generate maps. Section 5 concludes the paper and discusses possibilities for future experimentation.

2 Fungal Physiology

This section provides an overview of the natural process on which the fungal search algorithm is built.

Fungi are a highly diverse group of organisms, and many fungi may not conform to this strategy, or may follow it to varying degrees. In fact this algorithm is based on the strategy of basidiomycetous fungi, which often grow in maze-like structures in nature, such as wood [1].

Hyphae (plural; singular is *hypha*) are the tubular structures created by fungal organisms. Collectively, the hyphae form a network known as the *mycelium*. This network is always growing as the fungus seeks out nutrients. Many fungi also have the ability to produce *spores*, which are then spread by other mediums, such as air or water, and begin growing their own network of hyphae if they encounter favorable conditions. This allows the fungus to spread to areas that the original organism may be unable to reach.

The algorithm based on this biological process is defined in the next section.

3 Fungal Search Algorithm

This section describes the fungal search algorithm proposed in [2] for use in maze exploration.

Initially, a number of artificial spores (*AS*) are generated. Each spore checks its surrounding area for artificial nutrients (*AN*). If nutrients are found, the spore generates artificial hyphae (*AH*) to connect it to the nutrients. If no nutrients are found near a spore, the spore goes “dormant,” i.e. it is not checked again by the algorithm. Each hypha performs a similar search in its surrounding area for AN, and if any are found, it generates additional hyphae to connect the spore to those nutrients, and so on. If no nutrients are found near a hypha, the hypha generates a new spore near its endpoint.

3.1 Pseudocode

Pseudocode for the fungal search algorithm is given in Algorithm 1. This pseudocode comes directly from Koen and Engelbrecht’s work [2].

3.2 Complexity

Koen and Engelbrecht show that the complexity of the algorithm is $O(m^3)$, m being the number of hyphae, which is usually an order of magnitude greater than n , the number of spores [2].

4 Fungal Map Generation

Koen and Engelbrecht’s paper explores the concept of a fungal search algorithm for maze exploration. I had noticed that many maze searching algorithms can also be used to generate mazes. For example, Koen and Engelbrecht used a depth-first search maze generator to produce the mazes they used to test their algorithm [2].

Building off the pseudocode provided by Koen and Engelbrecht [2], which is here given in Algorithm 1, I implemented in C++ a version of the fungal search algorithm that generates random maps, with the intent to explore whether this can be further developed to generate solvable mazes. This section discusses this undertaking.

4.1 Adjustments

The ability of the artificial spores to spread depends largely on the availability of artificial nutrients in the testing environment. To construct a map, we want the fungus to spread over as much of the map as possible, and yet to avoid, for the most part, collisions with itself.

To ensure the artificial fungus spreads over as much of the map as possible, every pixel of the map is initially set to contain an artificial nutrient.

When a hypha is spawned, the nutrients are removed from all pixels within a specified radius; this assures that the hyphae, which will form the maze walls, leave enough space between them for the player to move through the maze.

4.2 Discoveries

A high initial spore count, though it serves to ensure the artificial fungus fills the map as much as possible, results in an unsolvable maze. Spores are generally a central point for multiple hyphae, and thus a source of “dead ends” in a maze. Too many dead ends, and the maze becomes impassable. Figure 1 shows a map generated with a high initial spore count.

Lowering the initial spore count reveals that when given a wide expanse of high-nutrient area with few obstacles, chains of hyphae will often eventually curl inwards, creating barriers for themselves that result in premature termination. Thus, we get incomplete mazes such as that shown in Figure 2. Note that although the fungus does not adequately cover the map, it is clearly closer to a solvable maze than Figure 1, having less frequent collisions between maze walls.

The problem of prematurely ended hypha chains can be somewhat mitigated by increasing the area in which new spores can spawn around a “dead” hypha. Recall that biological fungi release spores in order to travel where the original organism may not be able to reach; so too can artificial spores travel beyond the reach of the dead-ended artificial hyphae. When we raise the seed radius, we enable new spores to essentially “hop” over whatever is blocking the hyphae from progressing further. Figure 3 shows a map generated by a low initial spore count, but a higher seed radius. Note that although it still does not fully fill the map, it comes closer than Figure 2, yet retains its passability as a maze. However, if this happens too often, we may end up with too many spores, and once again generate only unsolvable mazes.

4.3 Accessing and Running the Code

The source code for fungal map generation is publicly available on github: <https://github.com/elotil/Fungus.git>

First, clone or download the git repository. Once in the directory, type **make run** in the terminal to compile and run the code. During execution of the code, you will be asked for a number of inputs. These values are explained below.

1. *Board size*. The dimension of the square board in which the map will be generated. For example, if you enter 20, the program will generate a map 20 pixels high and 20 pixels wide.
2. *Initial spores*. The number of spores generated at the beginning of the algorithm. See Section 4.2 for details on how this value affects the outcome.
3. *Search radius*. The radius around each spore or hypha tip that will be searched for new nutrients.
4. *Block radius*. The radius around each new hypha that will be emptied of nutrients. I recommend this value be set to something smaller than both the search radius and the seed radius.
5. *Seed radius*. The radius around hypha tips wherein new spores may be generated. See Section 4.2 for details on how this value affects the outcome.
6. *Print fungus maze*. If selected, this will print to the terminal an ASCII representation of fungus locations. An **x** represents the presence of a spore or hypha; a blank space represents an absence of fungus. This option is not recommended for large board sizes.
7. *Print blocks*. If selected, this will print to terminal an ASCII representation of nutrient locations. An **x** represents an absence of nutrients; a blank space represents the presence of nutrients. This option is mainly intended for debugging purposes; however, it may be interesting to some users, and so it was left in. Again, this is not recommended for large board sizes.
8. *Save to file*. Saves a bitmap representation of the map to file. A black pixel represents the presence of a spore or hypha; a white pixel represents an absence of fungus. On a successful file write, the name of the file will be printed. The file will have the **.pbm** file type; this is a very simplistic image format. If you are unable to open the file, [this tool](#) can be used to view it in your web browser. Several sample output files are also included in the project.

5 Conclusion and Future Work

The fungal search algorithm is obviously very flexible. After developing the fungal map generation program, I am confident that the algorithm can be further

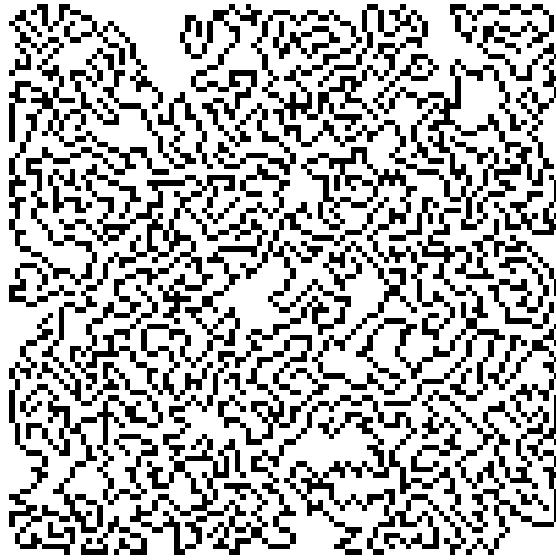


Figure 1: An example of a map generated by too many spores.

tailored to generate only solvable mazes. It is clear that with some tweaking of the parameters already available in the program, the algorithm can be made to generate fungi that encompass most of the map while still leaving space between the walls. The question is how to ensure that the result always contains a passable route from start to finish.

For future implementations, I hope to experiment with initial spore placement. With the current algorithm, spores are randomly placed throughout the map during initialization. Future implementations will use set configurations, such as setting all initial spores along one edge of the map, to see if this results in fewer collisions and thus a more navigable map.

I would also like to explore the idea of variable hypha behavior depending on environment. Currently, a hypha has only two options: spawn another hypha (if there are nutrients nearby) or spawn a spore (if there are no nutrients nearby). In future implementations, I may try to devise a way for a hypha chain to detect that it is curling inward on itself, and attempt to correct this.



Figure 2: An example of a map generated by too few spores.



Figure 3: An example of a map generated by few initial spores, but a higher seed radius.

```

create_initial_AS;
add_AS_to_seed_queue;
for AS in seed_queue do
    search_for_nutrients(AS, AN);
    if found(AS, AN) then
        | germinate(AS);
        | h = create_hypha(location(AS), location(AN));
        | add_to_mycelium_queue(h);
    end
    for AH in mycelium_queue do
        | if unexplored(location(AH)) then
        | | search_for_nutrients(location(AH), AN);
        | | if found(location(AH), AN) then
        | | | h = create_hypha(location(AH), location(AN));
        | | | add_to_mycelium_queue(h);
        | | else
        | | | if not(nutrients_nearby(location(AH))) then
        | | | | s = create_random_seed_near_hypha_tip(location(AH));
        | | | | add_to_seed_queue(s);
        | | | end
        | | end
        | end
    end
end

```

Algorithm 1: The FSA pseudocode.

References

- [1] Hanson, K., Nicolau, D., Filippini, L., Wang, L., Lee, A. and Nicolau, D. (2006), Fungi Use Efficient Algorithms for the Exploration of Microfluidic Networks. *Small*, 2: 1212-1220. doi:10.1002/sml.200600105
- [2] Koen, Renico and P. Engelbrecht, Andries. (2017). *Maze exploration using a fungal search algorithm: Part 1 - Algorithm model*. 40-45. 10.1145/3059336.3059365.